Budapest Parti Radar

Backend dokumentáció

Tartalom

1. Bevezetés 3
Mi az a backend? 3
A dokumentum felépítése 3
2. A backend helye a rendszerben 3
A backend a rendszer közepén 3
Egy példa: az „Ott leszek” gomb útja 4
Alapelvek 4
3. A szerver felépítése és indítása 4
Technológia 4
Mit csinál a szerver 4
Modulok 5
Beállítások és indítás 5
4. Regisztráció, belépés és jogosultságok 5
Regisztráció 5
Jelszavak védelme 5
Bejelentkezési tokenek 6
Mit lehet belépés nélkül, és mit belépve 6
5. Bulik, helyek és műfajok lekérése 6
A szerver által kiszámolt jelzések 7
A kereső 7
6. Az ajánló 7
7. Ott leszek és értékelés 7
Ott leszek 7
Értékelés 7
8. Buli beküldése és admin jóváhagyás 8
A beküldés útja 8
Admin végpontok 8
9. Értesítések, emlékeztetők és élő frissítés 9
Értesítések 9
24 órás emlékeztetők 9
Élő frissítés 9
10. Külső szolgáltatások 9
Időjárás 9
Buli-asszisztens 10
Automatikus adatbetöltés 10
11. Hibakezelés, biztonság, tesztelés és API leírás 10
Hibakezelés 10
Biztonság 10
Automatikus tesztek 11
API leírás (Swagger) 11
12. Munkafázisok és összefoglalás 11
Összefoglalás 11

1. Bevezetés

A Budapest Parti Radar egy weboldal, amely térképen mutatja meg, hol van ma este vagy a héten buli Budapesten. A projekt három részből áll, ebből a backendet, vagyis a szerveroldalt én, Zsombor készítem.

Ez a dokumentum csak az én részemről szól. Nem a felhasználónak szánt kezelési útmutató, hanem azt mutatja be, hogy mit építek a szerveroldalon, és hogyan fog működni.

Mi az a backend?

A backend a weboldal „háttere”: az a program, amely a szerveren fut. A felhasználó sosem látja, de minden kattintása hozzá fut be. Amikor valaki megnyitja a főoldalt, rányom az **Ott leszek** gombra, vagy kérdést ír a Buli-asszisztensnek, a böngésző kérést küld a szervernek. A szerver eldönti, hogy a kérés jogos-e, előveszi vagy elmenti a szükséges adatot az adatbázisban, majd választ küld vissza.

Egy hasonlat: ha az oldal egy étterem, akkor a frontend a terem és az étlap, az adatbázis a raktár, a backend pedig a konyha és a pincér együtt. Átveszi a rendelést, előkészíti a raktárból, ami kell, és kihozza a vendégnek.

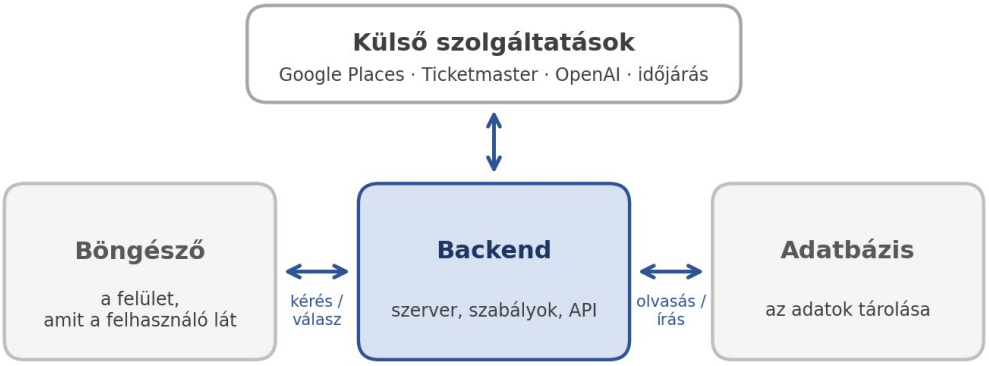
A dokumentum felépítése

A 2. fejezet a backend helyét mutatja a rendszerben, a 3. a szerver felépítését és indítását. A 4–10. fejezet sorra veszi a backend feladatait: a hitelesítést, a bulik lekérését, az ajánlót, a jelentkezést és értékelést, a beküldést és az admin jóváhagyást, az értesítéseket és a külső szolgáltatásokat. A 11. fejezet a hibakezelésről, a tesztelésről és az API leírásról szól, a 12. pedig a munkafázisokat foglalja össze.

2. A backend helye a rendszerben

A backend a rendszer közepén

A rendszer három rétegből áll: a böngészőben megjelenő felület, a szerver (ez az én részem) és az adatbázis. A felület nem tárol adatot, minden kérését a backendhez küldi, az adatbázishoz pedig csak a backend fér hozzá. Így minden szabályt egy helyen, a szerveren lehet betartatni.



A böngésző csak a backendet éri el; az adatbázishoz kizárólag a backend fér hozzá.

1. ábra A backend helye a rendszerben

Egy példa: az „Ott leszek” gomb útja

Az alábbi példán jól látszik, mit csinál a backend egyetlen kattintás alatt:

1. A böngészőből kérés érkezik a szerverhez: melyik bulira jelentkezik a felhasználó.
2. A backend ellenőrzi a belépést (a token alapján tudja, ki kéri), és azt, hogy a buli létezik-e.
3. Elmentem a jelentkezést az `attendances` táblába, vagy törlöm, ha a felhasználó visszavonta.
4. Megszámolom a jelentkezőket, és visszaküldöm az új számot.
5. Az élő frissítéssel minden nyitott oldalra elküldöm a változást, a hírszalagra pedig új hír kerül.
6. A buli előtt 24 órával a rendszer emlékeztető értesítést hoz létre a jelentkezőnek.

Ugyanez a minta ismétlődik minden funkciónál: **kérés, ellenőrzés, adatbázis, válasz**.

Alapelvek

- Az adatbázishoz csak a backend fér hozzá, a felület mindent tőlem kér le.
- A böngészőben történő ellenőrzések (például a jelszó erősségének mutatása) a felhasználót segítik, de a végső ellenőrzést mindig a szerver végzi el, mert a böngészőben bárki átírhat bármit.
- Ha a szabály nem teljesül (például hiányzik egy kötelező adat), érthető hibaüzenetet adok vissza a felhasználónak.

3. A szerver felépítése és indítása

Technológia

A backend **Node.js 22** és **Express** alapon készül. A Node.js futtatja a szervert, az Express pedig megkönnyíti az útvonalak, vagyis a végpontok megírását. Az adatokat egy MySQL vagy MariaDB adatbázisból olvassa és oda írja. A kód a `backend` mappában van: a `server.js` a szerver belépési pontja, mellette a témakörök szerinti modulok.

Mit csinál a szerver

- kiszolgálja a frontend oldalait (helyi gépen a `http://localhost:3000` címen);
- kiszolgálja az API végpontokat, amelyekre a frontend kérései érkeznek;
- kapcsolatot tart az adatbázissal, és induláskor létrehozza a hiányzó táblákat;
- időzített feladatokat futtat (például a 24 órás emlékeztetők);
- külső szolgáltatásokat hív (időjárás, Google Places, Ticketmaster, OpenAI).

Modulok

A kódot témakörök szerinti modulokra bontom, hogy egy-egy funkció egy helyen legyen, és könnyű legyen megtalálni, javítani és tesztelni. A pontos fájlnevek a fejlesztés során még változhatnak, a felosztás viszont ez lesz:

1. táblázat *A backend moduljai*

Modul	Feladata	Fejezet
Hitelesítés	regisztráció, belépés, tokenek, jogosultság-ellenőrzés	4.
Bulik és helyek	lekérés szűrőkkel, kereső, adatlapok	5.
Ajánló	a bulik pontozása a „Neked ajánljuk” sorhoz	6.
Jelentkezés és értékelés	Ott leszek, csillagos értékelés, szabályok	7.
Beküldés és admin	beküldött bulik, jóváhagyás, admin végpontok	8.
Értesítések és élő frissítés	értesítések, 24 órás emlékeztetők, hírszalag	9.
Külső szolgáltatások	időjárás, Buli-asszisztens, automatikus adatbetöltés	10.

Beállítások és indítás

A beállítások a `.env` fájlba kerülnek. A `.env.example` fájl mutatja, mit kell kitölteni, ezt kell `.env` néven lemásolni. Itt van például az adatbázis elérése és a külső szolgáltatások kulcsai (például az OpenAI kulcs). A titkos adatok így nem kerülnek bele a kódba, és a böngészőhöz sem jutnak el.

Az indítás saját gépen:

1. A XAMPP vezérlőpultban elindítom a MySQL-t.
2. Első alkalommal a phpMyAdmin SQL fülén lefuttatom az `adatbazis/schema.sql`, majd az `adatbazis/seed.sql` tartalmát. Később ezt nem kell megismételni, mert az új táblákat a szerver induláskor magától létrehozza.
3. A `backend` mappában a `.env.example` fájlt lemásolom `.env` néven.
4. Parancssorból kiadom az `npm install`, majd az `npm start` parancsot.
5. A böngészőben megnyitom a `http://localhost:3000` címet.

Ha egy régebbi verzió még fut, előbb le kell állítani (`taskkill /F /IM node.exe`), és frissítés után a böngészőben Ctrl+F5-tel érdemes újratölteni az oldalt.

4. Regisztráció, belépés és jogosultságok

Regisztráció

A regisztrációnál a szerver minden adatot újra ellenőriz, akkor is, ha a felület már megtette:

- a felhasználónév 3–30 karakter lehet, és csak betűt, számot, pontot és alulvonást tartalmazhat;
- az e-mail cím érvényes formátumú legyen;
- a jelszó legalább 8 karakter legyen;
- a felhasználónév és az e-mail cím egyedi: ha már foglalt, a szerver ezt jelzi.

Sikeres regisztráció után a fiók létrejön, a felhasználó pedig azonnal be is van léptetve.

Jelszavak védelme

A jelszót sosem tárolom olvasható formában. Mentés előtt egyirányú titkosítással (hash) alakítom át, így az adatbázisból a jelszó nem fejthető vissza, még akkor sem, ha valaki hozzáférne az adatokhoz. Belépéskor a megadott jelszóval elvégzem ugyanazt az átalakítást, és az eredményt a tároltal

hasonlítom össze. Ha a felhasználónév vagy a jelszó hibás, ugyanazt az üzenetet adom („Hibás felhasználónév vagy jelszó”), így kívülről nem derül ki, hogy melyik fiók létezik.

Bejelentkezési tokenek

Sikeres belépés után a szerver egy tokent ad a böngészőnek. A böngésző ezt küldi el minden további kérésnél, így tudom, ki kéri az adatot. A bejelentkezéseket a `sessions` tábla tartja nyilván. A felhasználói belépés 30 napig érvényes ugyanazon a böngészőn, az admin belépés viszont csak 8 óráig, mert annak nagyobb a jogköre. Kilépéskor a token érvényét veszti.

Mit lehet belépés nélkül, és mit belépve

2. táblázat *Belépést igénylő és nem igénylő funkciók*

Funkció	Belépés kell?
Bulik böngészése, szűrők, kereső, hely oldala	nem
Élő hírszalag, időjárás, „Most népszerű” sor	nem
Buli-asszisztens	nem
„Neked ajánljuk” sor	igen
Ott leszek	igen
Értékelés	igen, és csak annak, aki jelentkezett a bulira
Bulijaim, értesítések, buli beküldése	igen
Admin felület	külön admin fiók

Ha valaki belépés nélkül védett funkciót próbál használni, a szerver elutasítja a kérést, és az „Ehhez be kell jelentkezned” üzenetet küldi vissza.

5. Bulik, helyek és műfajok lekérése

A főoldal lényege a bulik listája és a térkép. Ehhez a backend a következő lekéréseket szolgálja ki:

3. táblázat *A fő lekérések*

Lekérés	Mit ad vissza
Bulik listája	a „Most megy” és az „Ezen a héten” nézet bulijai napok szerint, helyükkel és műfajaikkal; szűrhető műfaj, kerület és nap szerint
Egy buli adatlapja	cím, hely, kezdés, vége, ár, műfajok, leírás, jegy link, jelentkezők száma, értékelések átlaga
Egy hely oldala	cím, fotó, átlagos értékelés és a hely összes bulija
Kereső	találatok a buli neve, a hely vagy az előadó alapján; legalább 2 karakter kell hozzá
Műfajok, kerületek	a szűrők listái
Heti számok	a nyitó rész adatai: hány buli, hány jelentkező és hány hely van a héten

A szűrőket az adatbázis lekérdezésében alkalmazom, nem a böngészőben, így csak a szükséges adat utazik. A szűrők együtt is működnek, például „techno, VII. kerület, szombat”. Minden szűrőkombinációhoz egy végpont tartozik, amely az adatbázistól kapott adatot rendezett formában adja tovább.

A szerver által kiszámolt jelzések

A kártyákon látszó jelzéseket a backend számolja ki, így a felületnek nem kell számolnia, csak kiírja, amit kap:

- **MOST MEGY**: a kezdés már elmúlt, a vége még nem;
- **Népszerű**: legalább 3-an jelentkeztek;
- **Nem ellenőrzött**: az admin még nem hagyta jóvá az adatot;
- **Kerti idő ma este**: a hely kerthelyiséges, és az esti időjárás jó (lásd a 10. fejezetet);
- a jelentkezők száma, valamint az értékelések átlaga és száma.

A kereső

A kereső gépelés közben is használható, ezért gyorsnak kell lennie. A kérés rövid, a válasz csak a legfontosabb adatokat tartalmazza, az 1 karakteres keresést pedig el sem indítom.

6. Az ajánló

A **Neked ajánljuk** sor személyre szabott. A backend minden közelgő bulira pontot számol, majd a legtöbb pontot kapottakat küldi vissza. Csak közelgő vagy éppen zajló bulikat pontozok, a már véget értek nem kerülnek be.

4. táblázat *Az ajánló pontozási szempontjai*

Szempont	Hatása
A buli műfaja egyezik a felhasználó kedvenc műfajával	pontot ad
Népszerűség (a jelentkezők száma)	minél többen mennek, annál több pont

A két szempont pontjait összeadom, és a legtöbb ponttal rendelkező bulik kerülnek a sorba. Minden kártyához indokot is küldök (például „Szereted: House”), hogy a felhasználó lássa, miért kapta az ajánlást. A pontok súlyait kipróbálás után még finomíthatom.

Belépés nélkül nincs kedvenc műfaj, ezért ilyenkor csak a népszerűség számít: a szerver a **Most népszerű** sort adja vissza.

7. Ott leszek és értékelés

Ott leszek

- A jelentkezés ki-be kapcsolható: ugyanaz a gomb jelentkezik, és visszavon.
- Egy felhasználó egy bulira csak egyszer jelentkezhet, és ehhez belépés kell.
- A jelentkezők száma azonnal frissül mindenkinél, újratöltés nélkül (lásd a 9. fejezetet).
- Jelentkezéskor új hír kerül a hírszalagra.
- A buli előtt 24 órával a jelentkező emlékeztetőt kap.

Értékelés

Az értékelés szabályait a szerver tartatja be, mert itt a legkönnyebb visszaélni: például valaki nem járt a bulin, mégis értékelne.

5. táblázat *Az értékelés szabályai*

Szabály	Mit ellenőrzök
Csak az értékelhet, aki jelentkezett	van-e Ott leszek jelentkezés az adott bulira
Csak a buli vége után lehet értékelni	a buli vége már elmúlt-e
1–5 csillag adható	az érték egész szám 1 és 5 között
A szöveges rész nem kötelező	a csillag a szöveg nélkül is érvényes
Egy felhasználó egy bulit egyszer értékel	újabb küldés a meglévő értékelést módosítja

Az átlagot és az értékelések számát minden adatlapnál és a hely oldalán is kiszámolom. Ha valamelyik szabály nem teljesül, érthető hibaüzenetet küldök vissza, amelyet a felület kiír.

8. Buli beküldése és admin jóváhagyás

A beküldés útja

1. A belépett felhasználó kitölti az űrlapot: a hely, a név és a kezdés kötelező, a vége, a belépő, a jegy link és a leírás nem. Kiválasztja a műfajokat is.
2. A szerver ellenőrzi az adatokat: létezik-e a hely, a kezdés nem múltbeli-e, és a belépő nem negatív-e (0 = ingyenes).
3. Elmentem a bulit „Jóváhagyásra vár” állapottal. Ilyenkor csak a beküldőnek látszik, a **Beküldéseim** listában.
4. Az admin jóváhagyja vagy elutasítja.
5. Jóváhagyás után a buli „Kint van az oldalon”, és mindenki látja. A beküldő a döntésről értesítést kap.

6. táblázat *A beküldött buli állapotai*

Állapot	Ki látja	Jelentése
Jóváhagyásra vár	a beküldő és az admin	még nem nézte meg senki
Kint van az oldalon (Ellenőrzött)	mindenki	az admin jóváhagyta
Elutasítva	a beküldő	az admin elutasította, erről értesítést kap

Admin végpontok

Az admin felülethez minden adatot és műveletet a backend ad. Az admin külön fiókkal lép be (`admins` tábla), és az admin végpontok csak érvényes admin tokennel érhetők el. A belépés 8 óráig érvényes.

7. táblázat *Az admin felület backend oldala*

Rész	Amit a backend nyújt
Statisztika	összesítések az adatbázisból: műfajok népszerűsége, események kerületenként, melyik napon van a legtöbb buli, a top 5 közelgő buli
Importok	a Google Places, a Ticketmaster és az OpenAI adatbetöltés kézi indítása, és a korábbi futások naplója
Új esemény	buli felvétele kézzel (hely, név és kezdés kötelező)
Események	az összes buli listája, szerkesztés, törlés és jóváhagyás; törléskor a hozzá tartozó jelentkezések is törlődnek

9. Értesítések, emlékeztetők és élő frissítés

Értesítések

Az értesítések a `notifications` táblába kerülnek. A harang melletti számhoz az olvasatlanok számát adom, a lenyíló listához pedig az értesítéseket. Egy elemre kattintva a felhasználó oda jut, ahová az értesítés mutat, ezért minden értesítéshez eltárolom a célját is. Értesítést két esetben küldök:

- 24 órán belül kezdődik egy buli, amelyre a felhasználó jelentkezett;
- az admin jóváhagyja vagy elutasítja a beküldött bulit.

24 órás emlékeztetők

Egy időzített feladat rendszeres időközönként megnézi, mely jelentkezett bulik kezdődnek 24 órán belül, és létrehozza hozzájuk az értesítést. Minden jelentkezéshez csak egy emlékeztető készül, hogy a felhasználó ne kapjon többször ugyanarról.

Élő frissítés

A hírszalag és a jelentkezők száma újratöltés nélkül frissül. Ehhez a böngésző nyitva tart egy kapcsolatot a szerverrel, a szerver pedig elküldi az új eseményeket: ki jelentkezett, ki értékelt, milyen új buli került fel. A hírre kattintva a buli adatlapja nyílik meg, ezért minden hírhez elküldöm a buli azonosítóját is.

10. Külső szolgáltatások

Időjárás

A nyitó rész jobb felső sarkában az esti, 22:00-ra szóló időjárás látszik. Ezt a backend kéri le egy külső időjárás-szolgáltatástól, így egy helyen kezelhető. Ha meleg és száraz este várható, a szerver megjelöli a **Kerti idő** jelzést, és ezt a kerthelyiséggel rendelkező bulikra is érvényesíti. Ha a szolgáltatás nem érhető el, az időjárás egyszerűen nem jelenik meg, az oldal többi része viszont működik tovább.

Buli-asszisztens

Az asszisztens a felhasználó szabad szavas kérdésére a következő 7 nap bulijai közül ajánl néhányat. A működése:

- a kérdés a backendhez érkezik, belépés nélkül is;
- ha a szerveren be van állítva az OpenAI kulcs, mesterséges intelligencia értelmezi a kérdést;
- kulcs nélkül kulcsszavas keresés fut: a műfaj, a kerület, az ár és a nap szavakat érti a legjobban;
- a válasz szöveget és bulikártyákat tartalmaz, amelyekre kattintva megnyílik az adatlap;
- percenként legfeljebb 8 kérdést fogadok el, hogy ne lehessen túlterhelni a szolgáltatást.

A kulcs csak a szerveren van meg, a böngésző sosem látja.

Automatikus adatbetöltés

Hogy az oldalon mindig legyen friss tartalom, a backend külső forrásokból is tölt be adatot: a Google Places elsősorban a helyek adatait (cím, koordináta, fotó) hozza, a Ticketmaster és az OpenAI alapú import pedig bulikat. A betöltés időzítve és az admin felületről kézzel is indítható. Egy futás lépései:

1. Lekérem az adatokat a külső szolgáltatástól.
2. Átalakítom őket az adatbázis formájára (hely, kezdés, műfajok, forrás).
3. Kiszűröm a duplikátumokat, így ugyanaz a buli nem kerül be kétszer.
4. Az újakat „Jóváhagyásra vár” állapotban mentem el, hogy az admin ellenőrizze őket.
5. A futásról bejegyzés készül az `import_runs` naplótáblában, amelyet az admin látni tud.

11. Hibakezelés, biztonság, tesztelés és API leírás

Hibakezelés

A szerver minden hibára érthető magyar üzenetet és megfelelő HTTP állapotkódot ad, a felület pedig ezt írja ki. A leggyakoribb üzenetek:

8. táblázat *A leggyakoribb hibaüzenetek*

Üzenet	Mikor jön
Ez a felhasználónév vagy e-mail cím már foglalt	regisztrációkor, ha már van ilyen fiók
A jelszó legalább 8 karakter legyen	túl rövid jelszó esetén
Hibás felhasználónév vagy jelszó	sikertelen belépéskor
Ehhez be kell jelentkezned	belépést igénylő funkciónál, token nélkül

Ha a szerver egyáltalán nem fut, a felület az „Valami hiba történt. Fut a szerver?” üzenetet mutatja. Ilyenkor a `backend` mappában az `npm start` paranccsal kell elindítani, és ellenőrizni, hogy a MySQL fut-e a XAMPP-ban.

Biztonság

- a jelszavakat csak titkosítva tárolom, a belépéshez tokent használok;
- minden bejövő adatot ellenőrzök, és az adatbázis-lekérdezéseket paraméterezetten futtatom, hogy ne lehessen rosszindulatú kódot beszúrni;
- a külső szolgáltatások kulcsai csak a `.env` fájlban vannak;
- az asszisztensnél kérdéskorlát van, az admin belépés pedig rövidebb ideig érvényes.

Automatikus tesztek

A fontos funkciókra automatikus teszteket írok, és minden nagyobb változtatás után lefuttatom őket, hogy a régi funkciók ne romoljanak el. A tesztek elsősorban ezeket fedik le:

- regisztráció és belépés (jó és hibás adatokkal is);
- a jogosultságok: mit lehet belépés nélkül, és mit nem;
- az Ott leszek ki-be kapcsolása és a jelentkezők számának helyessége;
- az értékelés szabályai (csak jelentkezett, csak a buli vége után);
- a beküldés és az admin jóváhagyás.

API leírás (Swagger)

Az összes végpontot Swagger leírásban dokumentálom: látszik, mi a végpont címe, milyen adatot vár, mit ad vissza, és kell-e hozzá belépés. Ez a leírás a végpontok pontos, kipróbálható dokumentációja.

12. Munkafázisok és összefoglalás

A munkát úgy sorolom, hogy mindig az épüljön előbb, amire a többi funkció támaszkodik:

9. táblázat *A backend munkafázisai*

#	Munkafázis	Fejezet
1.	Alap szerver, oldalak kiszolgálása, adatbázis-kapcsolat	3.
2.	Regisztráció, belépés, tokenek, jogosultságok	4.
3.	Bulik, helyek, műfajok lekérése, szűrők és kereső	5.
4.	Az ajánló pontozása	6.
5.	Ott leszek és értékelés	7.
6.	Buli beküldése, admin jóváhagyás, admin végpontok	8.
7.	Értesítések, emlékeztetők, élő frissítés	9.
8.	Időjárás, Buli-asszisztens, automatikus adatbetöltés	10.
9.	Hibakezelés, automatikus tesztek és Swagger API leírás	11.

Összefoglalás

A backend a Budapest Parti Radar középső rétege: a felhasználó kérései és az adatok között áll. Az én feladatom, hogy az adatok biztonságosan, a megfelelő szabályok szerint jussanak el a felhasználóhoz, és vissza. Ide tartozik a regisztráció és a belépés, a bulik lekérése és ajánlása, a jelentkezés és az értékelés, a beküldés és az admin jóváhagyás, az értesítések, az élő frissítés, az időjárás, a Buli-asszisztens és az automatikus adatbetöltés, valamint a tesztek és az API leírás.